

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP LỚN

NỀN TẢNG AI TẠO SINH

ĐỀ TÀI

**Phân tích cảm xúc theo khía cạnh và sinh báo cáo tổng hợp
từ phản hồi khách hàng bằng Generative AI**

Giảng viên hướng dẫn: TS. Dương Quang Huy

Nhóm thực hiện: Nhóm 15

Trần Hoàng Sơn – 20252606M

Trần Mạnh Đạt – 20252607M

Nguyễn Đức Hoàng – 20252603M

Trần Hữu Vinh – 20252027M

Phạm Khánh Hưng – 20252249M

Mục lục

Danh mục viết tắt	2
Tóm tắt	3
1 Giới thiệu	4
1.1 Chủ đề và động lực	4
1.2 Vì sao chọn hướng tiếp cận generative	4
1.3 Pipeline tổng quan	6
1.4 Tổng quan các phương pháp	6
2 Công trình liên quan	7
2.1 Aspect-Based Sentiment Analysis	7
2.2 Repo nền: Generative-ABSA	7
2.3 Dataset: SemEval Restaurant Triplet data	7
2.4 Mô hình sinh văn bản: T5 và FLAN-T5	7
2.5 Kiểm tra tính xác thực (factual checking)	7
3 Bài toán	9
4 Phương pháp	10
4.1 Hai baseline so sánh: AsteLookupBaseline và GTS-BERT	10
4.2 T5-small / T5-base / FLAN-T5-base fine-tuned cho ASTE	11
4.3 Tổng hợp thống kê theo khía cạnh + top-10 lý do	12
4.4 Sinh báo cáo bằng FLAN-T5	13
4.5 Factual checker	14
4.6 Triển khai	15
5 Kết quả và đánh giá	17
5.1 So sánh 2 baseline vs 3 model T5	17
5.2 Chất lượng bảng tổng hợp aspect + lý do	17
5.3 Demo quy mô lớn – Yelp Restaurant Reviews	18
5.4 Đánh giá factual checker	18
5.5 Error analysis	21
6 Khám phá mới / Thảo luận	22
7 Hạn chế	23
8 Kết luận	24
Phân công nhóm	25
A Phụ lục	26
A.1 Bảng hyperparameter đầy đủ	26
A.2 Ví dụ output thực tế	26
A.3 Ví dụ báo cáo tự động sinh ra	26
Mã nguồn và notebook	27
Tài liệu tham khảo	28

Danh mục viết tắt

Viết tắt	Ý nghĩa
ABSA	Aspect-Based Sentiment Analysis
ASTE	Aspect Sentiment Triplet Extraction
AOPE	Aspect Opinion Pair Extraction
UABSA	Unified Aspect-Based Sentiment Analysis
TASD	Target Aspect Sentiment Detection
GAS	Generative Aspect-based Sentiment analysis (khung phương pháp)
GTS	Grid Tagging Scheme
T5	Text-to-Text Transfer Transformer
FLAN-T5	Fine-tuned LAnguage Net – T5 (phiên bản T5 đã instruction-tuned)
BERT	Bidirectional Encoder Representations from Transformers
NLP	Natural Language Processing
POS	Part-of-Speech
CLI	Command Line Interface
GPU	Graphics Processing Unit
F1	F1-score (trung bình điều hoà giữa Precision và Recall)

Bảng 1: Danh mục các từ viết tắt sử dụng trong báo cáo.

Tóm tắt

Thay vì chỉ phân loại một review là tích cực/tiêu cực, hệ thống trong báo cáo này xác định cụ thể khách hàng đang khen/chê điểm nào (khía cạnh – *aspect*) **trực tiếp từ câu thô, không cần biết trước khía cạnh**, kèm luôn “lý do” (*opinion*) cho mỗi nhận định, rồi tổng hợp thống kê trên nhiều review và dùng mô hình sinh văn bản để tạo báo cáo tóm tắt ngắn, kèm một bộ kiểm tra tính xác thực (factual checker) để hạn chế mô hình bịa số liệu.

Nhóm sinh trực tiếp bộ ba (khía cạnh, ý kiến, sentiment) từ câu thô – bài toán **Aspect Sentiment Triplet Extraction** (ASTE) – trên domain nhà hàng (SemEval Restaurant Triplet data, 14res+15res+16res), so sánh một baseline tra cứu tần suất tự xây với ba checkpoint T5-family (T5-small, T5-base, FLAN-T5-base) cùng hyperparameter. T5-base cải thiện gần gấp đôi triplet-F1 so với baseline ($0.3736 \rightarrow 0.7442$) và được chọn làm model chính. Nhóm cũng ghi lại một số phát hiện đáng chú ý trong quá trình thực nghiệm, trong đó có việc FLAN-T5-base chưa hội tụ trong ngân sách 20 epoch được cấp, gợi ý khoảng cách hiệu năng đo được có thể phản ánh tốc độ hội tụ chứ không hẳn năng lực model.

1 Giới thiệu

1.1 Chủ đề và động lực

Các hệ thống phân tích cảm xúc truyền thống thường chỉ đưa ra một nhãn tích cực/tiêu cực duy nhất cho toàn bộ review, trong khi một review thực tế thường đề cập nhiều khía cạnh khác nhau với sentiment khác nhau. Với người quản lý nhà hàng, một con số tổng quát (“70% review tích cực”) không đủ để biết *nên cải thiện điểm gì* – họ cần biết cụ thể khía cạnh nào đang được khen, khía cạnh nào đang bị chê, và vì sao. Đây chính là động lực của bài toán *Aspect-Based Sentiment Analysis* (ABSA), minh hoạ qua ví dụ:

“The food was great but the service was slow.”

Khía cạnh	Sentiment	Lý do (opinion)
food	Tích cực	great
service	Tiêu cực	slow

Bảng 2: Ví dụ phân tích cảm xúc theo khía cạnh (Aspect-Based Sentiment Analysis).

Khi có hàng trăm, hàng nghìn review như vậy, việc đọc thủ công từng câu là không khả thi. Nhóm hướng tới một hệ thống tự động hoá toàn bộ quy trình: từ review thô, tự tổng hợp và sinh ra một đoạn báo cáo dạng:

“Khách hàng đánh giá tích cực về food (đa số nhắc ‘great’, ‘delicious’) và price. Tuy nhiên, service là khía cạnh có nhiều phản hồi tiêu cực nhất (thường bị chê ‘slow’) và cần được ưu tiên cải thiện.”

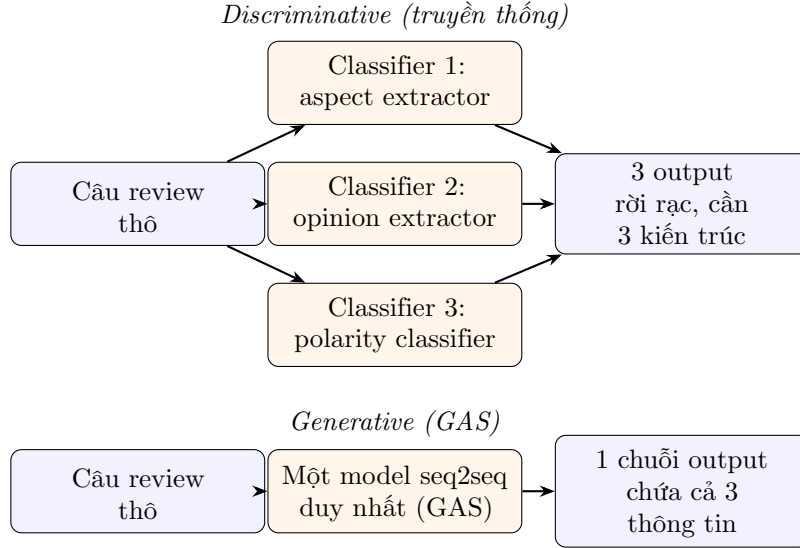
Một rủi ro lớn khi dùng mô hình sinh văn bản (generative model) cho bước cuối là hiện tượng *hallucination* – mô hình có thể bịa ra số liệu hoặc nhận định không khớp với dữ liệu gốc. Vì vậy pipeline của nhóm có thêm một bước kiểm tra tính xác thực (factual checker) trước khi xuất báo cáo cuối cùng.

1.2 Vì sao chọn hướng tiếp cận generative

Các phương pháp ABSA truyền thống xử lý theo hướng *discriminative* (phân loại) – với mỗi bài toán con (trích aspect, phân loại sentiment, ...), thiết kế một mạng phân loại riêng, dùng chỉ số lớp (class index) làm nhãn huấn luyện. Zhang et al. [2] chỉ ra cách này có hai hạn chế: (1) bỏ qua ngữ nghĩa tự nhiên của nhãn (*label semantics*) – ví dụ hiểu nghĩa cụm “food quality” giúp mô hình liên kết tốt hơn với aspect cụ thể như “pizza”, so với chỉ dùng một chỉ số lớp trừu tượng không mang thông tin; (2) mỗi bài toán ABSA cần một kiến trúc/thiết kế riêng, khó tái sử dụng giữa các bài toán.

Giải pháp – khung GAS (Generative Aspect-based Sentiment analysis): nhóm tác giả bài báo “Towards Generative Aspect-Based Sentiment Analysis” (Zhang, Li, Deng, Bing, Lam – ACL 2021) [2] đề xuất chuyển toàn bộ các bài toán ABSA thành bài toán sinh văn bản (text generation), dùng chung một mô hình seq2seq (**T5-base**), với hai kiểu định dạng target (*paradigm*):

- **Annotation-style:** chèn nhãn trực tiếp vào ngay trong câu gốc, bọc trong ngoặc vuông [aspect | opinion | polarity]. Ví dụ (task AOPE): input “Salads were fantastic, our server was also very helpful.” → target “[Salads | fantastic] were fantastic here, our [server | helpful] was also helpful.”



Hình 1: So sánh hướng discriminative (nhiều classifier rời rạc, mỗi bài toán con một kiến trúc riêng) và hướng generative theo khung GAS (một model seq2seq duy nhất, sinh trực tiếp toàn bộ thông tin dưới dạng chuỗi văn bản có cấu trúc).

- **Extraction-style:** sinh trực tiếp các cặp/bộ ba dưới dạng (aspect, opinion, polarity), tách nhau bởi dấu ;. Cùng ví dụ trên $\rightarrow$ target (Salads, fantastic); (server, helpful).

Đây chính là cơ sở cho định dạng target nhóm dùng ở Mục 4.2 (aspect: X | opinion: Y | sentiment: Z ; ...) – một biến thể gần với extraction-style, có thêm nhãn trường tường minh (aspect:/opinion:/sentiment:) để dễ parse bằng regex hơn là chỉ dựa vào thứ tự vị trí trong ngoặc.

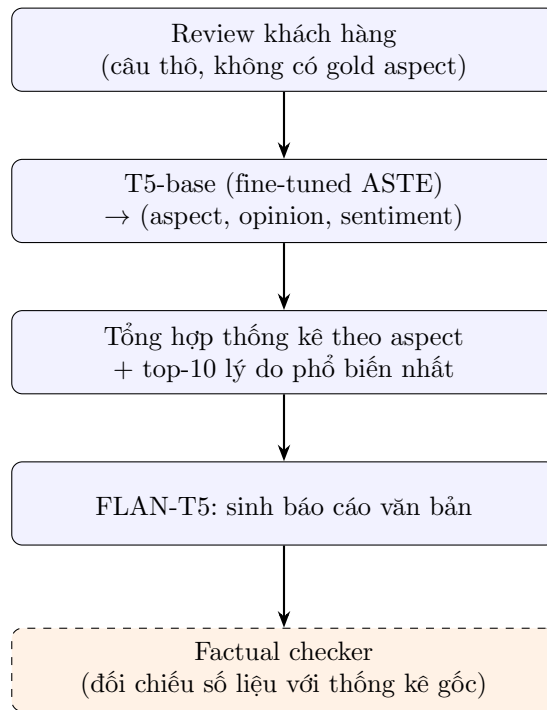
Bốn bài toán ABSA được bài báo thử nghiệm:

- **AOPE** (Aspect Opinion Pair Extraction) – trích cặp (aspect, opinion), chưa có sentiment.
- **UABSA** (Unified ABSA, còn gọi End-to-End ABSA) – trích đồng thời (aspect, sentiment), không cần biết trước aspect.
- **ASTE** (Aspect Sentiment Triplet Extraction) – trích bộ ba (aspect, opinion, sentiment) – đầy đủ nhất trong 4 task về số thành phần trích xuất.
- **TASD** (Target Aspect Sentiment Detection) – trích bộ ba (aspect, category, sentiment), trong đó category thuộc một tập cố định định trước (ví dụ “food quality”, “restaurant general”), có xử lý cả trường hợp aspect ẩn (implicit, gán nhãn null).

Vì sao nhóm chọn ASTE: so với 3 task còn lại, ASTE khớp nhất với động lực đã nêu ở Mục 1.1 – vừa giải quyết bài toán trích xuất aspect từ câu thô (giống UABSA/AOPE, không cần biết trước aspect như ABSA truyền thống), vừa cho thêm trường *opinion* (giống AOPE, mà UABSA không có) để làm “lý do” phục vụ trực tiếp bước sinh báo cáo (Mục 4.4) – đúng nhu cầu báo cáo cần giải thích được *lý do*, không chỉ đưa ra con số. TASD tuy đầy đủ nhất về mặt phân loại (aspect category cố định) nhưng lại đòi hỏi một tập category định sẵn theo domain – không phù hợp với mục tiêu trích xuất khía cạnh *tự do*, không giới hạn trước của nhóm. Nhóm chọn **IsakZhang/Generative-ABSA** [2] – code đi kèm bài báo trên – làm repo nền để kế thừa ý tưởng (chi tiết ở Mục 2.2).

1.3 Pipeline tổng quan

Hình 2 minh họa toàn bộ pipeline: một model duy nhất (T5-base fine-tune cho ASTE) đọc câu thô và sinh trực tiếp cả ba thông tin (aspect, opinion, sentiment) – không cần một bước trích xuất aspect riêng biệt như cách tiếp cận ABSA truyền thống (phân loại sentiment khi đã biết trước aspect).



Hình 2: Pipeline tổng quan. Một model duy nhất (T5-base fine-tune cho ASTE) đọc câu thô và sinh trực tiếp cả 3 thông tin (aspect, opinion, sentiment) – không cần bước trích xuất aspect riêng biệt. Khối nét đứt (factual checker) là phần do nhóm Hoàng/Vinh/Hưng phụ trách.

1.4 Tổng quan các phương pháp

Nhóm xây dựng và so sánh nhiều phương pháp theo từng bước của pipeline, được trình bày chi tiết ở Mục 4:

- **Baseline tra cứu tần suất (AsteLookupBaseline)** – mốc so sánh phi-neural cho bài toán trích xuất bộ ba.
- **T5-small / T5-base / FLAN-T5-base fine-tuned cho ASTE** – sinh trực tiếp bộ ba (khía cạnh, ý kiến, sentiment) từ câu thô.
- **FLAN-T5 sinh báo cáo** (đã fine-tune) từ bảng thống kê + reason khía cạnh (Mục 4.4).
- **Factual checker** đối chiếu số liệu và reason trong báo cáo sinh ra với bảng thống kê gốc (Mục 4.5).

Về phạm vi báo cáo: nhóm còn khảo sát một hướng khác (phân loại sentiment theo khía cạnh đã biết trước, TF-IDF+LogReg → DistilBERT/BERT, domain laptop, SemEval-2014 Task 4) như một bước thăm dò ban đầu – code, notebook và kết quả vẫn còn trong repo (notebooks/finetune_{distilbert,bert}_semeval_laptop.ipynb, plans/project-plan.md) nhưng không đưa vào báo cáo này để giữ báo cáo tập trung vào một pipeline hoàn chỉnh, nhất quán.

2 Công trình liên quan

2.1 Aspect-Based Sentiment Analysis

ABSA thường được chia thành các bài toán con: *Aspect Term Extraction* (tìm cụm từ chỉ khía cạnh trong câu), *Aspect Term Polarity* (phân loại sentiment cho từng aspect term đã tìm được), và ở mức chi tiết hơn – *opinion term extraction* (tìm cụm từ diễn đạt lý do) [1]. Aspect Sentiment Triplet Extraction (ASTE) là bài toán hợp nhất cả ba: sinh ra bộ ba (aspect, opinion, sentiment) trong một bước duy nhất, thay vì pipeline nhiều bước tách rời.

2.2 Repo nền: Generative-ABSA

Nhóm chọn **IsakZhang/Generative-ABSA** [2] – code cho bài báo “**Towards Generative Aspect-Based Sentiment Analysis**” (Zhang, Li, Deng, Bing, Lam – ACL 2021) – làm repo nền để kế thừa ý tưởng. Động lực chọn hướng generative, khung GAS (2 paradigm annotation-style/extraction-style), 4 bài toán ABSA bài báo thử nghiệm (AOPE/UABSA/ASTE/ TASD), và lý do nhóm chọn cụ thể ASTE trong 4 bài toán đó đã trình bày ở Mục 1.2.

Đáng chú ý: hyperparameter mặc định của repo này (`lr=3e-4`, `batch_size=16`, `20 epoch`) trùng khớp với config nhóm đã chọn độc lập cho các notebook fine-tune T5 – một tín hiệu cho thấy lựa chọn hyperparameter của nhóm phù hợp với chuẩn đã được kiểm chứng trong literature.

Điểm khác của nhóm so với repo nền: Generative-ABSA dừng lại ở việc sinh bộ ba cho từng câu riêng lẻ (đúng phạm vi một bài báo NLP). Nhóm mở rộng thêm: (1) bước tổng hợp thống kê + xếp hạng top-10 lý do phổ biến nhất theo tần suất trên *nhiều* review; (2) một baseline tra cứu tần suất tự xây (không có trong bài báo gốc) làm mốc so sánh phi-neural; (3) module sinh report bằng FLAN-T5 từ bảng thống kê; (4) một factual checker để kiểm tra report có bịa số liệu hay không; (5) một demo quy mô lớn trên dữ liệu Yelp thực tế, không có gold label.

2.3 Dataset: SemEval Restaurant Triplet data

Nhóm dùng bộ dữ liệu SemEval Restaurant Triplet data [3, 4], cụ thể phiên bản **ASTE-Data-V1-AAAI2020**, với cách gán nhãn theo nhóm token (mỗi aspect gán một mã nhóm, opinion tương ứng gán mã nhóm cùng độ dài dạng `S...S`) để một câu có thể chứa nhiều bộ ba độc lập.

2.4 Mô hình sinh văn bản: T5 và FLAN-T5

T5 [5] (Text-to-Text Transfer Transformer) đóng khung mọi tác vụ NLP dưới dạng text-to-text, phù hợp tự nhiên với bài toán sinh bộ ba có cấu trúc từ câu thơ. **FLAN-T5** [6] là phiên bản T5 đã được instruction-tuned trên nhiều tác vụ đa dạng, đã cho kết quả tốt trên các benchmark table-to-text/data-to-text như ToTTo [7] mà không cần fine-tune sâu – đây là lý do nhóm chọn FLAN-T5 (thay vì T5 gốc) cho bước sinh báo cáo cuối, dù kết quả thực nghiệm lại cho thấy FLAN-T5-base *không* phải lựa chọn tốt hơn T5-base cho bước trích xuất bộ ba (Mục 6) – một phát hiện thú vị được thảo luận riêng.

2.5 Kiểm tra tính xác thực (factual checking)

Trong literature có nhiều hướng phức tạp để kiểm tra tính nhất quán của văn bản sinh ra so với nguồn dữ liệu gốc, ví dụ FactCC [8] (train một classifier riêng để phát hiện câu không nhất quán với văn bản nguồn) hay các phương pháp dùng model riêng để đối chiếu theo từng claim (ConFactCheck và các hướng liên quan [10]). Các hướng này cần huấn luyện thêm một model kiểm tra riêng, vượt quá phạm vi và thời gian của môn học. Nhóm chọn cách tiếp cận rule-based nhẹ hơn nhưng vẫn xử lý được văn xuôi tự do (không ép một mẫu câu cố định): trích mọi con số

xuất hiện trong report bằng regex, gán mỗi con số về đúng aspect được nhắc gần nó nhất, rồi đối chiếu trực tiếp với bảng thống kê gốc – chi tiết cụ thể ở Mục 4.5.

3 Bài toán

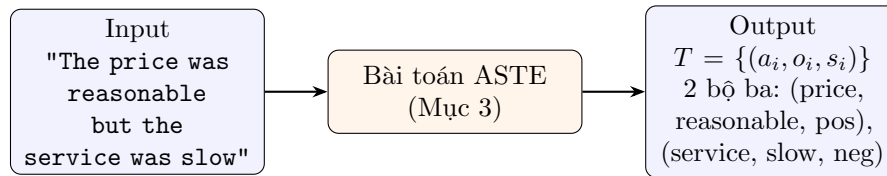
- **Input:** một câu review thô (tiếng Anh), *không* có thông tin khía cạnh đi kèm.
- **Output:** danh sách bộ ba (*aspect, opinion, sentiment*), ví dụ:

aspect: price | opinion: reasonable | sentiment: positive

Trường *opinion* chính là cụm từ diễn đạt lý do (reason) cho sentiment đó, phục vụ trực tiếp cho bước sinh báo cáo.

- **Dataset:** SemEval Restaurant Triplet data (14res + 15res + 16res) – do dữ liệu chỉ thuộc miền nhà hàng, kết quả của hệ thống chủ yếu phản ánh năng lực xử lý review nhà hàng, chưa thể suy rộng trực tiếp sang các miền khác (khách sạn, sản phẩm điện tử, vận chuyển, ...) nếu không có đánh giá bổ sung.

Về mặt hình thức, với mỗi câu review đầu vào x , hệ thống cần sinh ra một tập bộ ba $T = \{(a_i, o_i, s_i)\}_{i=1}^n$, trong đó a_i là aspect term, o_i là opinion term, $s_i \in \{\text{positive, negative, neutral}\}$, và n – số bộ ba trong câu – không cố định mà phụ thuộc nội dung từng câu (có thể bằng 0 nếu câu không chứa đánh giá rõ ràng).



Hình 3: Luồng input/output của bài toán ASTE: từ một câu thô, không có gold aspect, sinh ra trực tiếp một tập bộ ba (aspect, opinion, sentiment), số lượng bộ ba không cố định.

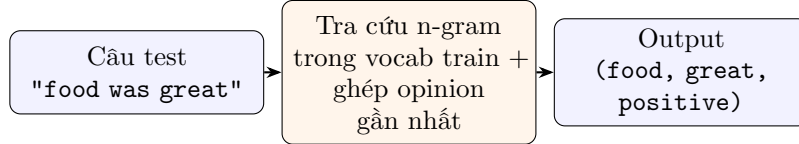
Bước tổng hợp và sinh báo cáo (sau khi có bộ ba trên toàn bộ tập review):

- **Input:** danh sách (aspect, opinion, sentiment) trên toàn bộ tập review.
- **Output:** (1) bảng thống kê theo từng aspect (số lượt positive/negative/neutral, top-10 lý do phổ biến nhất mỗi sentiment), (2) một đoạn báo cáo văn bản ngắn tóm tắt xu hướng, (3) kết quả kiểm tra tính xác thực của báo cáo đó.

4 Phương pháp

4.1 Hai baseline so sánh: AsteLookupBaseline và GTS-BERT

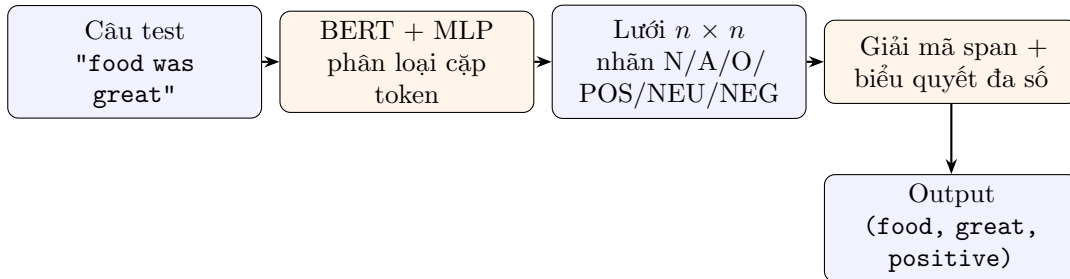
ASTE không có một baseline TF-IDF/Logistic-Regression tương đương (bài toán cần tự tìm aspect, không phải phân loại khi đã biết trước) – nhóm xây một baseline phi-neural để làm mốc so sánh: không cần train gradient, chỉ ghi nhớ (a) tập các cụm aspect và (b) sentiment phổ biến nhất của từng cụm opinion, cả hai đều thống kê trực tiếp từ tập train. Ở câu test, thuật toán tra cứu nguyên văn các cụm đã thấy trong câu (khớp n-gram dài nhất trước, tối đa 3 từ, quét trái sang phải không chồng lấn), rồi ghép mỗi aspect tìm được với opinion *gần nhất* trên câu (theo khoảng cách token).



Hình 4: Luồng input/output của AsteLookupBaseline: không có bước học gradient, chỉ tra cứu + ghép cặp theo thống kê từ tập train.

Baseline này có recall thấp theo thiết kế: bất kỳ cách diễn đạt aspect hay opinion nào khác với những gì đã thấy trong tập train đều bị bỏ sót hoàn toàn – đây chính là “sàn” để đo mức độ cải thiện thực sự của một model biết tổng quát hoá. Cài đặt: `src/baseline/aste_lookup_baseline.py`, chạy hoàn toàn local (không cần GPU) qua `scripts/eval_aste_baseline.py`.

Baseline neural thứ hai – GTS-BERT. AsteLookupBaseline là sản phẩm phi-neural; để có thêm một mốc so sánh *neural nhưng không generative* (khác hẳn cách tiếp cận T5), nhóm cài đặt lại Grid Tagging Scheme (GTS, Wu et al., Findings of EMNLP 2020) [9]: với câu có n từ, gán nhãn cho mọi cặp từ (w_i, w_j) , $i \leq j$, một trong 6 nhãn {N, A, O, POS, NEU, NEG} – A/O đánh dấu hai từ cùng thuộc một aspect/opinion span, POS/NEU/NEG đánh dấu một từ thuộc aspect, từ kia thuộc opinion, và cặp đó mang sentiment tương ứng. BERT-base-uncased encode câu, một MLP phân loại mọi cặp token thành lưới $n \times n$; bộ ba được giải mã lại bằng cách đọc các đoạn liên tục cùng nhãn trên đường chéo (aspect/opinion span), rồi biểu quyết đa số nhãn sentiment giữa các ô nổi mỗi cặp span. Đây là bản cài đặt lại độc lập dựa theo mô tả trong bài báo (không phải code gốc của tác giả) – cách xây biểu diễn cặp r_{ij} cụ thể ($\text{concat}(h_i, h_j, h_i \odot h_j)$ qua MLP) là lựa chọn của nhóm vì bài báo không mô tả đủ chi tiết công thức attention layer của họ. Hyperparameter: 20 epoch, $\text{lr}=2\text{e-}5$ (mức chuẩn để fine-tune toàn bộ BERT, khác hẳn $\text{lr}=3\text{e-}4$ dùng cho T5 – không dùng chung vì hai kiến trúc cần tốc độ học khác nhau), batch size 16.



Hình 5: Luồng input/output của GTS-BERT: khác AsteLookupBaseline (tra cứu) và khác T5 (sinh văn bản) – đây là một bài toán phân loại có giám sát trên mọi cặp token trong câu.

Cài đặt: `src/data/gts_grid.py` (logic xây/giải mã lưới, có test local – `tests/test_gts_grid.py`, 10 test bao gồm một test tái hiện đúng ví dụ minh họa trong bài báo gốc) và `src/baseline/gts_bert_model.py` (model, cần GPU); train trên Kaggle qua `notebooks/train-gts-bert-for-aste-on-14res-15res-16res.ipynb`.

4.2 T5-small / T5-base / FLAN-T5-base fine-tuned cho ASTE

T5 là gì: T5 (Text-to-Text Transfer Transformer) [5] là một kiến trúc Transformer dạng *encoder-decoder*, được tiền huấn luyện (pretrain) trên corpus C4 bằng mục tiêu *denoising* kiểu điền-chỗ-trống (che ngẫu nhiên các đoạn văn bản, model học sinh lại đúng đoạn đã bị che). Điểm khác biệt của T5 so với các model chỉ-encoder như BERT là T5 đóng khung *mọi* tác vụ NLP (dịch, tóm tắt, phân loại, trích xuất, ...) dưới dạng “văn bản vào → văn bản ra”, nên có thể tái sử dụng cùng một kiến trúc cho bài toán sinh bộ ba có cấu trúc như ASTE mà không cần thiết kế tầng đầu ra riêng (khác với model phân loại cần một lớp softmax cố định số lớp).

T5-small vs. T5-base – khác nhau về quy mô: cả hai cùng kiến trúc, cùng cách pretrain, chỉ khác kích thước (Bảng 3). T5-base có nhiều tầng và chiều ẩn hơn T5-small gần 4 lần về số tham số, nên có năng lực biểu diễn (representational capacity) lớn hơn, thường học được các mẫu hình phức tạp/mơ hồ tốt hơn nhưng cần nhiều bộ nhớ GPU và thời gian train hơn.

FLAN-T5-base khác gì T5-base: FLAN-T5 [6] *không* phải một kiến trúc khác – FLAN-T5-base khởi tạo từ đúng checkpoint T5-base (cùng 220M tham số), nhưng được huấn luyện thêm một giai đoạn *instruction tuning*: fine-tune trên hơn 1800 tác vụ khác nhau được viết dưới dạng chỉ dẫn ngôn ngữ tự nhiên (Flan collection), trước khi nhóm fine-tune tiếp cho ASTE. Mục tiêu ban đầu của giai đoạn này là giúp model hiểu chỉ dẫn tốt hơn ở chế độ zero-shot/few-shot (lý do nhóm chọn FLAN-T5 cho bước sinh báo cáo, Mục 4.4) – nhưng cũng chính vì đã quen với văn phong đối thoại/chỉ dẫn tự nhiên, FLAN-T5-base cần nhiều epoch hơn để “quên” bớt thói quen đó và thích nghi với định dạng trích xuất terse của ASTE (`aspect: X | opinion: Y | ...`) – đúng như quan sát ở Mục 6.

Model	Tham số	Số tầng	d_{model}	Pretrain thêm
T5-small	60M	6	512	không
T5-base	220M	12	768	không
FLAN-T5-base	220M	12	768	instruction tuning (Flan)

Bảng 3: So sánh 3 checkpoint T5-family nhóm dùng cho ASTE. Số liệu kiến trúc theo Raffel et al. [5]; FLAN-T5-base cùng kiến trúc T5-base, chỉ khác ở việc được huấn luyện thêm giai đoạn instruction tuning trước khi nhóm fine-tune cho ASTE.

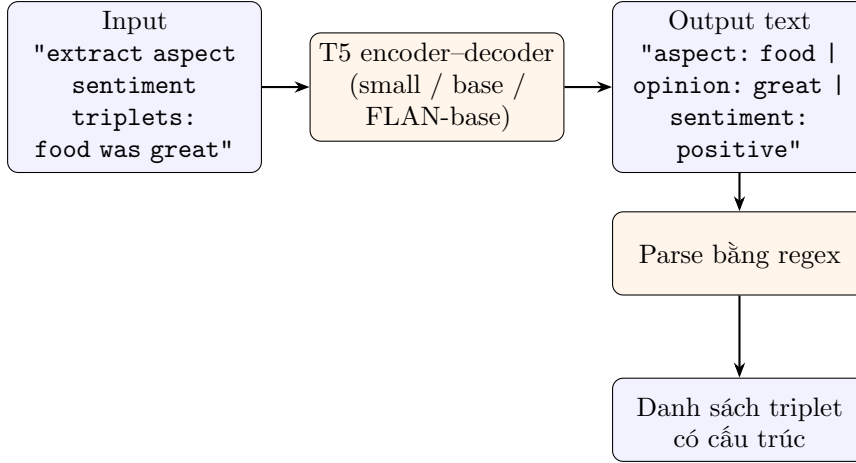
Nhóm sinh trực tiếp bộ ba (aspect, opinion, sentiment) bằng một mô hình text-to-text: câu thô được thêm một prefix hướng dẫn cố định ("`extract aspect sentiment triplets:` ") làm input, target text có định dạng "`aspect: X | opinion: Y | sentiment: Z ; ...`" (nói nhiều bộ ba bằng dấu ; nếu câu có nhiều aspect). Ví dụ với câu có 2 aspect, "*The pizza was delicious but the service was slow*", input đưa vào model (đã ghép prefix) và target tương ứng như sau:

```
extract aspect sentiment triplets: The pizza was delicious but
the service was slow.
```

```
aspect: pizza | opinion: delicious | sentiment: positive ;
aspect: service | opinion: slow | sentiment: negative
```

Nhờ đó cùng một model seq2seq học đồng thời việc phát hiện aspect, xác định opinion tương ứng, và gán sentiment – thay vì tách thành nhiều module rời rạc như cách tiếp cận discriminative truyền thống (Mục 1.2). Nhóm chọn định dạng có tên trường tương minh (`aspect:/opinion:/sentiment:`) thay vì chỉ dùng bộ ba trong ngoặc kiểu (`pizza, delicious, positive`) vì tên trường giúp giảm nhập nhằng khi hậu xử lý bằng regex, nhất là khi aspect hoặc opinion là cụm nhiều từ – không cần đoán trường nào ứng với vị trí nào trong ngoặc.

Hậu xử lý. Với mỗi đoạn khớp định dạng `aspect: X | opinion: Y | sentiment: Z`, hệ thống trích ba trường, chuẩn hoá về chữ thường và loại khoảng trắng dư thừa; nếu câu có nhiều bộ ba, tách theo dấu ;. Bước parse này quan trọng vì model generative có thể sinh



Hình 6: Luồng input/output khi fine-tune T5-family cho ASTE: cả 3 checkpoint dùng chung định dạng input/output, chỉ khác nhau ở kiến trúc bên trong (Bảng 3); chuỗi văn bản sinh ra được parse bằng regex thành danh sách bộ ba có cấu trúc.

sai cấu trúc quy ước – thiếu một trường, đảo thứ tự trường, sinh thêm ký tự thừa, hoặc gán sentiment ngoài tập nhãn hợp lệ – những đoạn không khớp định dạng bị loại, không tính là bộ ba hợp lệ. Vì vậy chất lượng model không chỉ phụ thuộc vào việc dự đoán đúng nội dung (đúng aspect/opinion/sentiment) mà còn phụ thuộc vào khả năng sinh đúng cấu trúc đã quy ước.

Ba checkpoint T5-family được fine-tune với *cùng* hyperparameter (20 epoch, lr=3e-4, effective batch size 16 – riêng T5-base dùng batch nhỏ hơn kèm gradient accumulation lớn hơn để giữ effective batch không đổi do cần nhiều bộ nhớ GPU hơn) để so sánh công bằng, chỉ đổi checkpoint: `t5-small`, `t5-base`, `google/flan-t5-base`.

Metric đánh giá là *triplet-F1*: Precision/Recall/F1 theo phép giao tập hợp (set overlap) giữa bộ ba dự đoán và bộ ba gốc (đã chuẩn hoá chữ thường), micro-average trên toàn bộ tập test (không average theo từng câu rồi lấy trung bình, để câu có nhiều bộ ba đóng góp đúng tỉ trọng). Đây là phép so khớp nghiêm ngặt ở cấp bộ ba hoàn chỉnh: một dự đoán chỉ tính là đúng khi cả 3 trường aspect, opinion, sentiment đều khớp – dự đoán đúng aspect và sentiment nhưng sai opinion vẫn bị tính là sai hoàn toàn, không được cộng điểm một phần. Hạ tầng: ba notebook Kaggle độc lập, tự động tải dataset (clone GitHub nếu chưa có sẵn trên Kaggle input) – `notebooks/train-{t5-small,t5-base,flan-t5-base}-for-aste-*.ipynb`.

Vì sao dùng T5-base cho ASTE nhưng FLAN-T5 cho sinh báo cáo – hai model, hai vai trò. Hai bước này có bản chất khác nhau. Trích xuất ASTE là bài toán sinh chuỗi bị ràng buộc chặt về định dạng: model phải sinh đúng aspect, đúng opinion, đúng sentiment, đúng cú pháp `aspect: ... | opinion: ... | sentiment: ...` thì mới parse tự động được (Mục 4.2 trên) – ở đây, một model đã *fine-tune chuyên biệt* trên chính định dạng đó (T5-base) có lợi thế hơn. Sinh báo cáo (Mục 4.4) ngược lại là bài toán *data-to-text*: input là bảng thống kê có cấu trúc, output là một đoạn văn tự nhiên, không có một cú pháp cứng nào để bám vào – ở bước này, khả năng làm theo chỉ dẫn tốt hơn nhờ instruction tuning của FLAN-T5 lại có lợi thế hơn. Vì vậy nhóm không coi FLAN-T5 là lựa chọn thay thế T5 trong toàn bộ pipeline, mà dùng mỗi model đúng vai trò nó mạnh nhất. Phát hiện ở Mục 6 (FLAN-T5-base thua T5-base trên chính bài toán trích xuất) càng củng cố lựa chọn kiến trúc này: instruction tuning giúp model tổng quát hoá tốt hơn cho các chỉ dẫn mở, nhưng không đảm bảo lợi thế trên một tác vụ sinh chuỗi ngắn, có cấu trúc nghiêm ngặt và được chấm bằng exact-match/F1 như ASTE.

4.3 Tổng hợp thống kê theo khía cạnh + top-10 lý do

Bước tổng hợp gộp các bộ ba (aspect, opinion, sentiment) theo aspect: đếm số lượt mỗi sentiment và lấy sentiment đa số; trong mỗi nhóm (aspect, sentiment), các cụm *opinion* được xếp hạng

theo tần suất, giữ lại top-10 làm ví dụ “lý do” minh họa cho sentiment đó – ví dụ aspect food với sentiment positive có thể có lý do phổ biến nhất là **great, good, delicious**.

Toàn bộ logic gộp số liệu này là thuần Python (không cần GPU), được viết và test local (`src/report/aspect_stats.py`, kiểm thử ở `tests/test_aspect_stats.py`); chỉ riêng phần suy luận mô hình (cần GPU) chạy trên Kaggle, sau đó logic tổng hợp đã test được inline nguyên văn vào notebook để đảm bảo đúng kết quả như bản đã kiểm thử local.

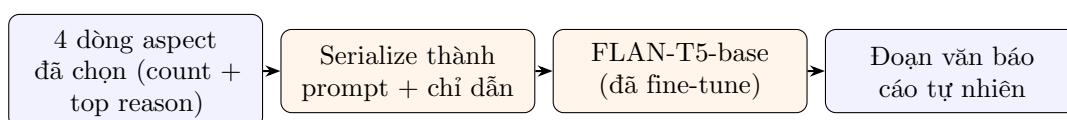
4.4 Sinh báo cáo bằng FLAN-T5

Chọn aspect đưa vào báo cáo. Từ bảng tổng hợp (Mục 4.3), hàm `select_report_rows` trước hết ưu tiên các aspect có ít nhất 10 lượt nhắc; nếu không có aspect nào thỏa mãn thì sử dụng toàn bộ bảng. Hệ thống chọn mặc định 4 aspect đại diện để đưa vào một báo cáo ngắn (đưa hết vài trăm aspect vào một đoạn văn là vô nghĩa): (1) aspect được nhắc nhiều nhất, (2) aspect có tỉ lệ positive/total cao nhất trong số 30 aspect được nhắc nhiều nhất (điểm mạnh rõ nhất), (3) aspect có tỉ lệ negative/total cao nhất (điểm yếu rõ nhất), (4) aspect có $|\text{positive} - \text{negative}|/\text{total}$ nhỏ nhất (ý kiến chia rẽ nhất) – chọn theo *tỉ lệ* chứ không theo số tuyệt đối để tránh việc luôn chọn trùng các aspect có tổng lượt nhắc lớn nhất cho cả 4 vai trò. Sau mỗi bước, aspect đã chọn được loại khỏi các bước tiếp theo, do đó bốn vai trò không bị gán cho cùng một aspect. Việc sử dụng tỉ lệ thay vì số tuyệt đối giúp tránh ưu tiên quá mức các aspect có tổng lượt nhắc lớn.

Prompt. Bảng 4 aspect đã chọn được serialize thành các dòng dạng sau, kèm chỉ dẫn yêu cầu model viết **một đoạn văn phân tích tự nhiên** (không ép đúng một mẫu câu cố định), so sánh các aspect, chỉ được dùng đúng số liệu và reason đã cho, không được bịa thêm sự kiện/phần trăm/tên món ăn/thương hiệu nào khác (`build_reasoned_prompt` trong `src/report/flan_t5_report.py`):

```
aspect=food | total=827 | positive=621 | negative=164 | neutral=42
positive_reasons=great (109), good (100)
negative_reasons=mediocre (10), bad (7)
```

Mặc dù bảng tổng hợp ở Mục 4.3 lưu tối đa 10 reason cho mỗi cặp (aspect, sentiment), prompt chỉ lấy tối đa hai reason phổ biến nhất của mỗi sentiment để giữ đầu vào ngắn gọn. Các reason của cùng một aspect được serialize trên cùng một dòng dữ liệu cùng với total và các sentiment count.



Hình 7: Luồng input/output khi sinh báo cáo: bảng thống kê → prompt → FLAN-T5 → đoạn văn, sau đó đưa tiếp qua factual checker (Mục 4.5).

Model. Nhóm fine-tune `google/flan-t5-base` trên các bảng aspect tổng hợp được sinh tổng hợp theo đúng cấu trúc dữ liệu đầu vào. Tập train, validation và test tổng hợp được tạo độc lập; tên aspect giữa các tập không trùng nhau và các aspect trong dữ liệu thật không được đưa vào quá trình huấn luyện. Bảng **predicted** thực tế chỉ được sử dụng để đánh giá cuối cùng, nhằm hạn chế rò rỉ dữ liệu. Toàn bộ quy trình được cài đặt trong notebook Colab tự chứa `notebooks/finetune_flan_t5_reasoned_report_colab.ipynb`.

Khi suy luận, mô hình sử dụng deterministic beam search với `num_beams=4` và `do_sample=False`, giúp kết quả có thể tái lập khi đầu vào và cấu hình không thay đổi. Hệ thống thực hiện tối đa 3 lượt sinh báo cáo, bao gồm một lượt sinh ban đầu và tối đa hai lượt thử lại. Mô hình và tokenizer chỉ được nạp một lần trước vòng lặp sinh. Nếu factual checker (Mục 4.5) từ chối kết quả của một lượt, prompt của lượt tiếp theo được bổ sung

chỉ dẫn Previous output was not fully grounded. Correct every aspect, count, and reason.

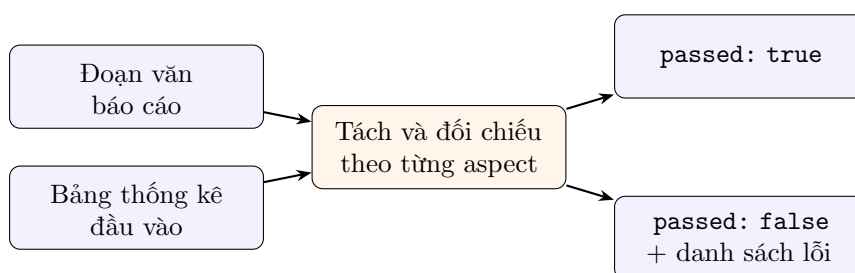
CLI:

```
python scripts/generate_report.py \
-model models/flan-t5-reasoned-report \
-output output/flan_t5_reasoned_report.json
```

Theo cấu hình mặc định, chương trình đọc bảng thống kê từ `output/aspect_reasons_restaurant.json` và sử dụng mảng `predicted`. Đây là bảng được xây dựng từ kết quả dự đoán của mô hình trích xuất T5-base ở Mục 4.2, thay vì nhãn `gold`, nên phản ánh đúng kịch bản triển khai thực tế. Bảng này chỉ được đưa vào mô hình sinh báo cáo ở giai đoạn suy luận và không được sử dụng để fine-tune FLAN-T5.

4.5 Factual checker

Do báo cáo được sinh dưới dạng văn xuôi tự do, không tuân theo một mẫu câu cố định, factual checker (`check_reasoned_report` trong `src/report/factual_checker.py`) không kiểm tra bằng cách khớp toàn bộ báo cáo với một biểu thức chính quy duy nhất như checker cho định dạng cố định ban đầu. Thay vào đó, hệ thống đối chiếu nội dung báo cáo với bảng thống kê đầu vào theo bốn bước sau:



Hình 8: Luồng input/output của factual checker. Checker nhận báo cáo và bảng thống kê đầu vào, sau đó trả về các cờ đánh giá, danh sách lỗi và cảnh báo. Khi kết quả không đạt theo chế độ kiểm tra đang sử dụng, hệ thống có thể đưa phản hồi này vào lượt sinh tiếp theo ở Mục 4.4.

- Tách báo cáo thành các câu và phân đoạn nhỏ hơn.** Trước hết, báo cáo được tách theo các dấu kết thúc câu (`.`, `!`, `?`). Mỗi câu tiếp tục được chia theo dấu chấm phẩy và một số cấu trúc chuyển ý, gồm `whereas`, `while` và `, but`. Bước này giúp xử lý trường hợp một câu đề cập đến nhiều aspect, chẳng hạn *“Indian food was praised, whereas dessert received 16 negative mentions.”*
- Liên kết mỗi phân đoạn với aspect tương ứng.** Checker tìm tên aspect xuất hiện dưới dạng cụm từ hoàn chỉnh trong từng phân đoạn. Các tên aspect dài hơn được ưu tiên trước khi xét các tên ngắn có thể nằm bên trong chúng. Chẳng hạn, `Indian food` được xét trước `food`, nhờ đó số liệu của `Indian food` không bị gán đồng thời cho `food`. Các phân đoạn được gán cho cùng một aspect sẽ được tổng hợp để thực hiện một lượt kiểm tra cho aspect đó.
- Đối chiếu số liệu.** Với mỗi aspect, checker xây dựng tập giá trị số được phép từ `total`, `positive`, `negative`, `neutral` và các số đếm reason có trong bảng thống kê. Mọi số xuất hiện trong các phân đoạn được gán cho aspect phải thuộc tập giá trị này. Một con số dù có tồn tại trong bảng nhưng thuộc aspect khác vẫn bị coi là không có căn cứ. Ngoài ra, phần nội dung của mỗi aspect phải nêu giá trị `total` và ít nhất một số đếm sentiment. Checker hiện kiểm tra giá trị số theo tập số liệu của aspect; nó chưa bắt buộc số đếm reason phải nằm ngay cạnh reason phrase tương ứng trong câu.

4. **Đối chiếu reason phrase.** Nếu bảng thống kê của một aspect chứa các reason, nội dung báo cáo về aspect đó phải đề cập đến ít nhất một reason phrase thuộc chính aspect này. Checker cũng phát hiện trường hợp báo cáo sử dụng reason chỉ thuộc về một aspect khác và trả về lỗi **reasons belong to another aspect**. Một reason xuất hiện hợp lệ ở nhiều aspect không bị xem là reason vay mượn.

Tách riêng tính đúng sự thật và độ bao phủ. Checker trả về hai cờ đánh giá:

- **factual_passed** cho biết các aspect đã được kiểm tra có sử dụng số liệu và reason phù hợp với bảng thống kê đầu vào hay không. Đây là tiêu chí chính để phát hiện thông tin không có căn cứ hoặc bị gán nhầm giữa các aspect.
- **coverage_passed** cho biết tất cả các aspect được chọn để đưa vào báo cáo có xuất hiện ít nhất một lần hay không. Cờ này chỉ phản ánh aspect bị thiếu; việc một aspect xuất hiện lặp lại được ghi nhận dưới dạng cảnh báo và không tự làm cho **coverage_passed** trở thành **false**.

Việc tách hai cờ giúp phân biệt báo cáo thiếu một phần nội dung nhưng các thông tin đã nêu đều đúng với báo cáo chứa số liệu hoặc reason sai. Checker hỗ trợ hai chế độ. Trong chế độ **factual-only** mặc định, **passed** nhận giá trị của **factual_passed**; aspect bị thiếu hoặc bị lặp được đưa vào **coverage_warnings** và không trực tiếp chặn kết quả. Trong chế độ **strict**, **passed** chỉ nhận giá trị **true** khi cả **factual_passed** và **coverage_passed** đều bằng **true**. Aspect xuất hiện lặp lại vẫn chỉ được xem là cảnh báo trong cả hai chế độ.

4.6 Triển khai

Ngôn ngữ và thư viện: toàn bộ code viết bằng Python 3.9+.

Thành phần	Thư viện / công cụ
Fine-tune & inference T5-family	transformers (HuggingFace), torch , sentencepiece
Baseline, tổng hợp thống kê	scikit-learn , numpy , scipy
Kiểm thử	pytest (45 test, xem bên dưới)
Hạ tầng chạy nặng	Kaggle notebook (GPU miễn phí)

Bảng 4: Ngôn ngữ và thư viện sử dụng.

Hạ tầng: tách rõ hai môi trường. Mọi bước cần GPU (fine-tune ba checkpoint T5-family; suy luận T5-base để tổng hợp aspect+reason; sinh report bằng FLAN-T5) chạy trên notebook Kaggle (GPU miễn phí). Mọi logic còn lại (parser dữ liệu, baseline tra cứu, hàm tổng hợp thống kê, factual checker, toàn bộ test suite) chạy local, không cần GPU – được viết và test đầy đủ ở local trước, sau đó mới inline nguyên văn vào notebook để chạy trên Kaggle, đảm bảo logic đã qua kiểm thử mới lên GPU.

Dựa trên code có sẵn nào, nhóm sửa đổi gì: như đã nêu ở Mục 2.2, nhóm không fork/clone trực tiếp repo nền IsakZhang/Generative-ABSA, mà tự cài đặt lại ý tưởng đóng khung ASTE dưới dạng sinh văn bản text-to-text bằng T5 (Mục 4.2), dùng chung dataset và hyperparameter mặc định của repo đó để so sánh công bằng. Các phần nhóm tự xây thêm, không có trong repo nền: baseline tra cứu tần suất (AsteLookupBaseline), bước tổng hợp thống kê + top-10 reason, module sinh report bằng FLAN-T5, factual checker, và demo quy mô lớn trên Yelp – liệt kê chi tiết ở Mục 2.2.

Cấu trúc repo và kiểm thử:

```
src/                                module Python tại su dung, co test
|-- data/                          parser XML / triplet format
```

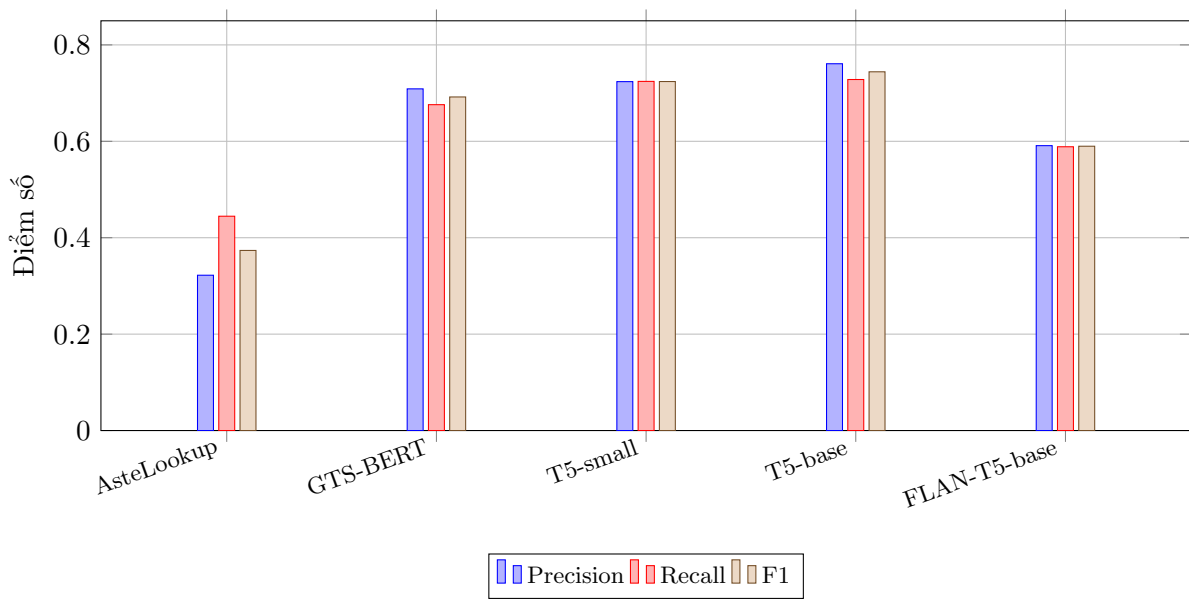
-- baseline/	TF-IDF+LogReg (tham do ban dau), AsteLookupBaseline
-- report/	tong hop thong ke, sinh report FLAN-T5, factual checker
scripts/	CLI chay local (train/eval baseline, sinh report)
notebooks/	template san sang chay tren Kaggle
notebooks-output/	ban da chay, co ket qua that
tests/	45 test pytest, toan bo chay local (khong can GPU)
output/, results/	ket qua / metrics da ghi lai, dung lam so lieu bao cao

5 Kết quả và đánh giá

5.1 So sánh 2 baseline vs 3 model T5

Model	Triplet-F1	Precision	Recall
AsteLookupBaseline (phi-neural)	0.3736	0.3222	0.4446
GTS-BERT (neural, không generative)	0.6920	0.7088	0.6760
T5-small	0.7240	0.7238	0.7243
T5-base	0.7442	0.7609	0.7282
FLAN-T5-base	0.5898	0.5910	0.5887

Bảng 5: So sánh trên cùng test split (1134 câu, 14res+15res+16res).

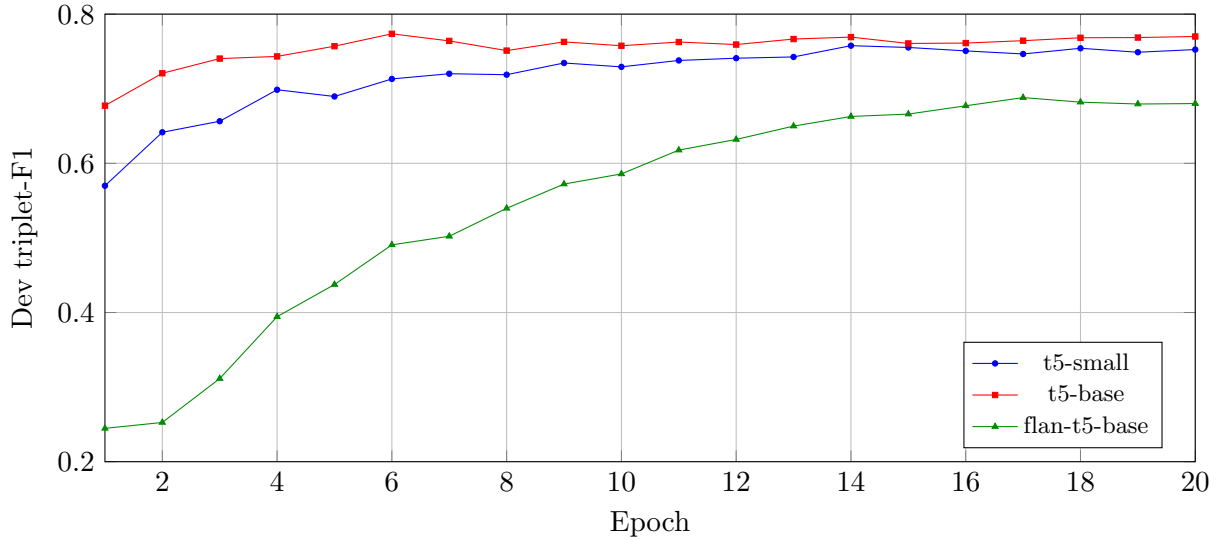


Hình 9: So sánh trực quan Precision/Recall/F1 giữa 5 phương pháp (cùng số liệu với Bảng 5).

T5-base cải thiện gần gấp đôi triplet-F1 so với baseline tra cứu ($0.3736 \rightarrow 0.7442$) và được chọn làm model chính. GTS-BERT – baseline neural nhưng không generative – đạt 0.6920, cao hơn hẳn baseline tra cứu và thậm chí cao hơn FLAN-T5-base, nhưng vẫn thấp hơn cả T5-small lẫn T5-base. Điều này cho thấy bản thân việc dùng neural network/BERT không phải là yếu tố quyết định khoảng cách hiệu năng – cách đóng khung bài toán (T5 sinh trực tiếp bộ ba dạng text-to-text, so với GTS-BERT phải phân loại lưới $n \times n$ rồi giải mã span) mới là khác biệt chính giúp hướng generative vượt trội hơn trên tác vụ này. Hình 10 cho thấy rõ lý do FLAN-T5-base thua kém: không phải nó “kém hơn” về bản chất mà đơn giản là chưa hội tụ trong ngân sách 20 epoch đã cấp. Xem Mục 6 để phân tích sâu hơn.

5.2 Chất lượng bảng tổng hợp aspect + lý do

Trên tập SemEval Restaurant có nhãn (4550 câu, train+dev+test), model T5-base cho **majority-sentiment agreement 97.25% (531/546)** giữa bảng tổng hợp từ gold triplets và bảng tổng hợp từ triplets model dự đoán (cùng một aspect, kết luận sentiment đa số có khớp không) – một mức tin cậy cao cho việc dùng bảng tổng hợp từ model làm input cho bước sinh báo cáo.



Hình 10: Dev triplet-F1 theo epoch, cả 3 lần chạy (số liệu lấy trực tiếp từ log training trong `notebooks-output/`). t5-small và t5-base hội tụ và bão hoà rõ trong khoảng epoch 10–15; đường flan-t5-base vẫn đang tăng dần đều tới hết epoch 20, chưa hề bão hoà – minh chứng trực quan cho phát hiện ở Mục 6: vấn đề là cần nhiều epoch hơn.

5.3 Demo quy mô lớn – Yelp Restaurant Reviews

Thay vì Amazon Reviews (dự kiến ban đầu, category Electronics – lệch domain hoàn toàn với model T5 train trên domain nhà hàng), nhóm đổi sang Yelp Restaurant Reviews để domain khớp với model, không có gold label (chỉ demo quy mô lớn, không benchmark).

Chạy trên 2000 review mẫu (seed=42) → 14285 câu (sau khi tách câu từ review nhiều câu) → 14700 bộ ba → 1224 aspect (≥ 2 lượt nhắc). Mẫu lấy được nghiêng nhiều về nhóm quán tráng miệng/bakery: top aspect ice cream (765 lượt, positive), place (633), flavors (237), cùng staff/service/ donuts/pastries/bakery – kết quả hợp lý, đúng domain.

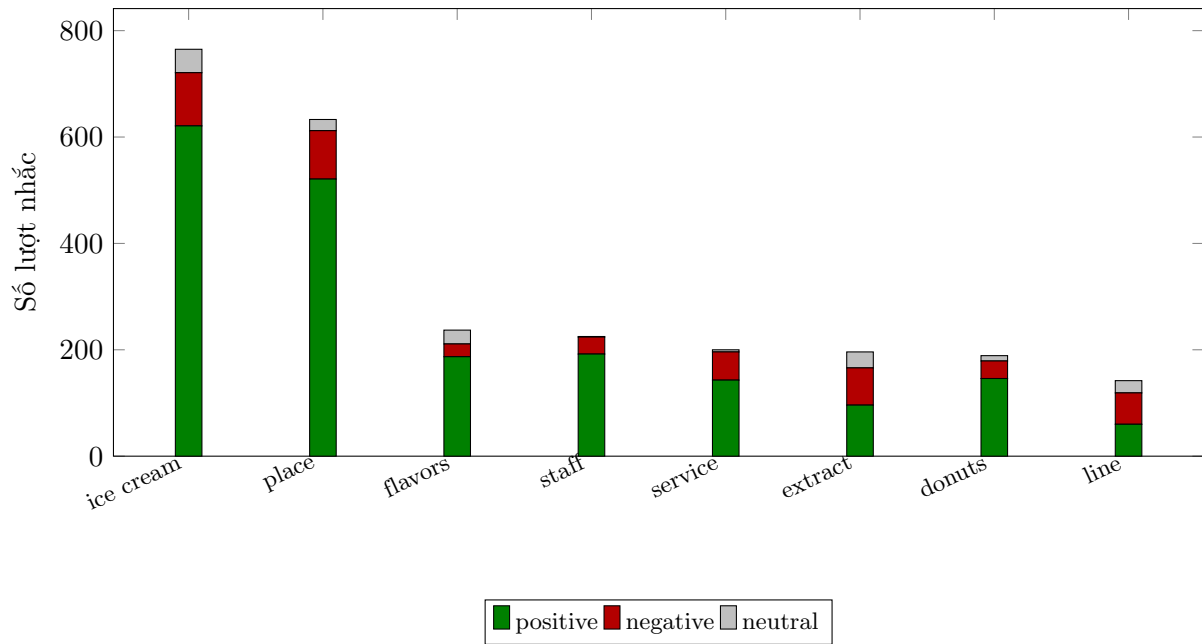
5.4 Đánh giá factual checker

Factual checker được đánh giá theo hai hướng: kiểm tra kết quả thực tế trên các báo cáo do FLAN-T5 sinh ra và kiểm thử đơn vị trên các trường hợp lỗi được xây dựng có chủ đích. Cần lưu ý rằng các kết quả dưới đây chủ yếu minh họa khả năng kiểm tra tính nhất quán của báo cáo với bảng thống kê đầu vào; chúng chưa phải là một benchmark đầy đủ để tính precision, recall hay F1-score của checker trên một tập báo cáo được gán nhãn thủ công.

Demo trên track nhà hàng. Track nhà hàng bắt nguồn từ bộ dữ liệu có nhãn, tuy nhiên trong kịch bản triển khai ở đây, factual checker không đối chiếu báo cáo với nhãn gold. Đầu vào của checker là mảng `predicted` trong `output/aspect_reasons_restaurant.json`, tức bảng thống kê được xây dựng từ kết quả dự đoán của mô hình trích xuất ở Mục 4.2. Vì vậy, kết quả kiểm tra cho biết báo cáo có bám sát bảng `predicted` đã cung cấp hay không, chứ không trực tiếp đánh giá độ chính xác của mô hình trích xuất so với nhãn chuẩn.

Từ bảng này, hệ thống chọn bốn aspect `food`, `indian food`, `dessert` và `waiter`. Báo cáo do mô hình FLAN-T5-base đã fine-tune sinh ra được lưu tại `output/flan_t5_reasoned_report.json`:

“The strongest takeaway is the contrast between Indian food and dessert. Indian food was praised in 21 of 22 mentions, especially for being great, whereas dessert received 16 negative comments out of 26, largely citing not inspired. At the same time, food remained the most visible topic, appearing 827 times with 621 positive mentions and



Hình 11: Top-8 aspect theo tổng lượt nhắc trong demo Yelp (bảng **predicted**, T5-base), tách theo sentiment. Aspect **extract** (hạng 6) là noise đáng chú ý, phân tích ở Mục 5.5.

great as a recurring reason. Feedback on waiter was less settled: 24 positive versus 18 negative reactions across 42 mentions, with experiences ranging from attentive to snobby.”

Checker trả về **factual_passed: true** vì các số liệu và reason được phát hiện trong phần nội dung của cả bốn aspect đều thuộc bảng thống kê tương ứng. Kết quả **coverage_passed: true** cho biết cả bốn aspect được chọn đều xuất hiện trong báo cáo. Hai aspect **dessert** và **indian food** được nhắc ở câu mở đầu, sau đó tiếp tục được trình bày trong câu chứa số liệu, nên checker ghi nhận cảnh báo lặp aspect. Cảnh báo này không biểu thị sai số liệu và không làm báo cáo bị từ chối trong chế độ **factual-only**.

Kết quả chi tiết như sau:

```
passed: true
factual_passed: true
coverage_passed: true
claims_checked: 4
valid_claims: 4
missing_aspects: []
repeated_aspects: [dessert, indian food]
status: accepted_with_warnings
```

Trong kết quả này, **claims_checked: 4** được hiểu là bốn aspect có ít nhất một phân đoạn được phát hiện và đưa vào kiểm tra. Chỉ số này không phải số lượng tất cả các phát biểu hoặc mệnh đề độc lập xuất hiện trong báo cáo. Tương tự, **valid_claims: 4** cho biết phần nội dung được tổng hợp theo cả bốn aspect đều vượt qua kiểm tra.

Demo trên track Yelp. Thử nghiệm tiếp theo sử dụng mảng **predicted** trong **output/aspect_reasons_yelp_demo.json**. Khác với track nhà hàng, bảng Yelp này không có nhãn **gold** tương ứng trong pipeline hiện tại. Vì vậy, checker chỉ đánh giá tính nhất quán giữa báo cáo và bảng thống kê dự đoán, không đánh giá bảng thống kê đó có phản ánh hoàn toàn chính xác dữ liệu Yelp ban đầu hay không.

Bốn aspect được chọn là **ice cream**, **atmosphere**, **parking** và **line**. Báo cáo được lưu tại **output/flan_t5_yelp_reasoned_report.json**:

“Customer feedback points to atmosphere as a standout: it drew 64 positive comments from 66 mentions, frequently because it was great. Although ice cream attracted the most discussion at 765 mentions and earned 621 positive reactions associated with best, not every area performed as well. In particular, parking generated 26 negative responses among 61 mentions, with limited recurring as the chief complaint. Meanwhile, parking split opinion: its 61 mentions contained 23 positive and 26 negative reactions, commonly framed as either plenty or limited.”

Báo cáo chỉ đề cập đến ba trong bốn aspect được chọn: **ice cream**, **atmosphere** và **parking**; aspect **line** bị bỏ sót. Aspect **parking** được nhắc trong hai phần khác nhau của báo cáo. Tuy nhiên, các số liệu và reason xuất hiện trong phần nội dung của ba aspect được phát hiện đều nằm trong tập giá trị hợp lệ của bảng thống kê tương ứng. Vì vậy, checker trả về **factual_passed: true** nhưng **coverage_passed: false**.

Kết quả chi tiết như sau:

```
passed: true
factual_passed: true
coverage_passed: false
claims_checked: 3
valid_claims: 3
missing_aspects: [line]
repeated_aspects: [parking]
status: accepted_with_warnings
```

Giá trị **passed: true** trong trường hợp này là kết quả của chế độ **factual-only**: báo cáo được chấp nhận vì những thông tin đã nêu đều bám sát bảng nguồn, còn việc thiếu **line** và lặp **parking** được lưu trong **coverage_warnings**. Nếu chạy ở chế độ **strict**, báo cáo này sẽ nhận **passed: false** do **coverage_passed: false**. Việc lặp **parking** vẫn chỉ được xem là cảnh báo; nguyên nhân trực tiếp làm kiểm tra độ bao phủ thất bại là aspect **line** bị thiếu.

Thử nghiệm Yelp cho thấy lợi ích của việc tách tính đúng sự thật khỏi độ bao phủ. Báo cáo không chứa số liệu hoặc reason bị checker xác định là sai, nhưng vẫn chưa hoàn thành đầy đủ yêu cầu sinh báo cáo do bỏ sót một aspect. Tuy nhiên, kết quả **factual_passed: true** chỉ khẳng định tính nhất quán với bảng **predicted**, không đủ để kết luận rằng toàn bộ nội dung là đúng so với dữ liệu Yelp gốc.

Kiểm thử đơn vị. File **tests/test_report_generation.py** hiện chứa 17 hàm kiểm thử. Các test bao phủ cả checker định dạng cố định ban đầu và checker reason-aware hiện tại. Đối với **check_reasoned_report**, các trường hợp quan trọng gồm phát hiện số liệu không thuộc aspect, reason được lấy từ aspect khác, aspect bị thiếu hoặc lặp, tên aspect có quan hệ chứa chuỗi như **food** và **Indian food**, câu chứa nhiều aspect được nối bằng liên từ tương phản, và sự khác nhau giữa hai cờ **factual_passed** và **coverage_passed**.

Do một số test trong file vẫn dành cho hàm **check_report** của định dạng báo cáo cố định trước đây, không nên hiểu rằng toàn bộ 17 test đều trực tiếp đánh giá checker văn xuôi reason-aware. Chẳng hạn, test từ chối phần văn bản thừa không chứa số liệu thuộc checker định dạng cũ và không chứng minh rằng checker mới có thể phát hiện mọi nhận định văn xuôi không có căn cứ.

Nhận xét. Hai lượt chạy thực tế cho thấy checker có thể xác nhận các số liệu và reason hợp lệ, phát hiện aspect bị thiếu, ghi nhận aspect bị lặp và phân biệt lỗi factual với lỗi coverage. Tuy nhiên, đây mới là đánh giá chức năng trên một số báo cáo cụ thể và các test case được thiết kế trước. Để đánh giá định lượng độ chính xác của checker, cần xây dựng thêm một tập báo cáo gồm cả mẫu đúng và mẫu chứa lỗi được gán nhãn thủ công, sau đó so sánh dự đoán pass/fail của checker với nhãn chuẩn để tính precision, recall, F1-score và confusion matrix.

5.5 Error analysis

Phân tích dưới đây dựa trực tiếp trên `output/aspect_reasons_restaurant.json` (bảng gold vs. predicted, T5-base, 4550 câu) và `output/aspect_reasons_yelp_demo.json` – không cần chạy lại model.

(1) 15 case majority-sentiment lệch giữa gold và predicted đều là các aspect “sát nút”, không lệch hệ thống. Trong 546 aspect xuất hiện ở cả hai bảng, 531 khớp kết luận sentiment cuối cùng (97.25%, Mục 5.2); 15 case còn lại gần như luôn là những aspect có số lượt positive/negative gần bằng nhau, chỉ cần lệch 1 câu là đổi phe đa số – ví dụ `cheese` (gold: 3 positive/6 negative → negative; predicted: 5/4 → positive), `dish` (gold 6/7 → negative; predicted 8/7 → positive), `noodles` (gold 0/1 trên 2 câu → negative; predicted 1/1 → positive), `japanese food` (gold 3/4 → negative; predicted 4/3 → positive). Không có case nào lệch mạnh (đa số rõ ràng ở gold nhưng đảo ngược hoàn toàn ở predicted) – cho thấy model không hề có thiên lệch sentiment hệ thống, sai số chỉ nằm ở các aspect vốn dĩ gây tranh cãi ngay cả với annotator.

(2) Model bỏ sót nhiều aspect hiếm, và có xu hướng đếm thiếu (dưới) trên hầu hết aspect phổ biến – khớp với Precision > Recall đã thấy ở Mục 5.1. 73 aspect chỉ có trong gold, không hề xuất hiện trong predicted – toàn bộ đều là aspect tần suất thấp (≤ 6 lượt nhắc, ví dụ `bathroom`, `flavor`, `ceiling`, `eggplant`), phần lớn tên món ăn hiếm/đặc thù mà model có thể chưa gặp đủ nhiều lúc train để tự tin trích ra. Ngược lại, trên các aspect phổ biến xuất hiện ở cả hai bảng, tổng lượt nhắc của predicted gần như luôn *thấp hơn* gold: `prices` (gold 69 → predicted 46, -23), `place` (278 → 258, -20), `food` (840 → 827, -13), `service` (568 → 562, -6). Đây là bằng chứng thứ hai, độc lập với Mục 5.1, cho cùng một hiện tượng: T5-base có Precision (0.7609) cao hơn Recall (0.7282) ở mức triplet – model “thận trọng”, thà bỏ sót một số triplet mơ hồ còn hơn đoán bừa.

(3) Aggregation theo chuỗi ký tự nguyên văn (exact-string) làm tách một aspect thành nhiều biến thể khác nhau – hạn chế của bước tổng hợp, không phải của model ASTE. Ví dụ rõ nhất: gold có aspect `rice to fish ration` (lỗi chính tả có sẵn trong review gốc), trong khi predicted trích đúng chính tả `rice to fish ratio` – vì bước tổng hợp (Mục 4.3) gom nhóm theo `aspect.strip().lower()` nguyên văn, hai chuỗi chỉ khác 1 ký tự này bị tính là 2 aspect hoàn toàn tách biệt thay vì được gộp làm một. Hiện tượng tương tự lặp lại ở dạng số ít/số nhiều: track nhà hàng có `place` (278 lượt, đã tính ở trên) tách biệt khỏi `places` (3 lượt, chỉ xuất hiện ở predicted); demo Yelp cho thấy mẫu hình này rõ hơn nhiều vì tập dữ liệu lớn hơn – `donuts` (189 lượt) tách khỏi `donut` (68 lượt), `cupcakes` (111) tách khỏi `cupcake` (59). Đây là hạn chế thật của cách tổng hợp (không lemmatize/chuẩn hoá hình thái), không phải lỗi của bước sinh triplet.

(4) Aspect noise trong demo Yelp: extract (196 lượt, hạng 6) – kiểm tra lại giả thuyết ban đầu. Ghi chú trước đó trong `plans/project-plan.md` suy đoán đây là phần bị cắt cụt từ “vanilla extract”. Kiểm tra trực tiếp các reason phrase gắn với aspect `extract` cho thấy giả thuyết này **không đứng vững**: reason phổ biến nhất là `good`(8), `worth`(6), `try`(5), `best`(3) – hoàn toàn chung chung, không có phrase nào liên quan tới vani/hương liệu, và một số reason còn vô nghĩa ở dạng cô lập (4., `huh`, `won't be coming back`). Nguyên nhân nhiều khả năng là model nhầm lẫn trên các câu ngắn/rời rạc, đặc thù của review Yelp quán tráng miệng (câu kiểu “Must try!”, “Worth it.”) khi bị tách câu – model không tìm được aspect thật trong câu và “bám” vào một token gần giống định dạng huấn luyện.

6 Khám phá mới / Thảo luận

1. **Đường cong chưa bão hoà không đồng nghĩa với model kém hơn.** FLAN-T5-base thua kém rõ rệt so với T5-base cùng kích thước trong cùng ngân sách 20 epoch, lr=3e-4 (0.5898 vs. 0.7442 triplet-F1). Tuy nhiên Hình 10 cho thấy `eval_triplet_f1` của flan-t5-base vẫn tăng dần đều tới hết epoch 20, chưa hề bão hoà – trong khi t5-small/t5-base đã bão hoà rõ từ epoch 10–15. Nhiều khả năng nguyên nhân không phải FLAN-T5-base “kém hơn” về bản chất, mà là model này (đã instruction-tuned trên nhiều task đa dạng) cần nhiều epoch hơn hẳn để thích nghi lại với định dạng trích xuất terse này – ngân sách 20 epoch đủ cho t5-small/t5-base nhưng nhiều khả năng chưa đủ cho flan-t5-base. Bài học: khoảng cách hiệu năng đo tại một điểm dừng cố định có thể phản ánh tốc độ hội tụ khác nhau hơn là năng lực thực sự của model. Kết quả này cũng củng cố lựa chọn kiến trúc hai-model-hai-vai-trò của nhóm (Mục 4.2): instruction tuning giúp FLAN-T5 tổng quát hoá tốt hơn cho các chỉ dẫn mới (phù hợp bước sinh báo cáo, Mục 4.4), nhưng không đảm bảo lợi thế trên một tác vụ sinh chuỗi ngắn, ràng buộc cú pháp chặt như ASTE – ở đây model fine-tune chuyên biệt (T5-base) vẫn thắng thế.
2. **Baseline tra cứu: precision thấp hơn recall – dấu hiệu lỗi nằm ở bước ghép cặp, không phải bước tra cứu.** AsteLookupBaseline có Precision 0.3222 thấp hơn hẳn Recall 0.4446 – ngược với trực giác thông thường về một baseline tra cứu nguyên văn (thường kỳ vọng precision cao, recall thấp: cái gì đã “chắc chắn thấy” thì đúng, chỉ là bỏ sót nhiều). Khả năng cao nguyên nhân nằm ở bước ghép cặp “aspect gần opinion nhất theo khoảng cách token”: trong câu có nhiều aspect/opinion xen kẽ, heuristic này dễ ghép nhầm một opinion với aspect không thực sự liên quan (dù bản thân aspect và opinion đều là cụm từ “đã biết” từ tập train), tạo ra nhiều bộ ba sai dạng false-positive. Mục 5.5 xác nhận đây thực sự là kiểu lỗi phổ biến ở mức aspect: nhiều aspect có tổng lượt nhắc bị đếm thiếu ở model T5-base cũng theo đúng hướng thận trọng-hơn-là-bịa này.

7 Hạn chế

- **AsteLookupBaseline giới hạn từ vựng nghiêm ngặt:** chỉ khớp nguyên văn cụm từ đã thấy trong train, không có bất kỳ hình thức chuẩn hoá ngữ nghĩa nào (đồng nghĩa, biến thể hình thái). Recall 0.4446 phần lớn phản ánh giới hạn này hơn là độ khó thực sự của bài toán.
- **Không dùng POS tagging thật** cho baseline (chỉ tra cứu + ghép theo khoảng cách token) – một baseline rule-based “chuẩn” hơn trong literature thường dùng POS tagger để tìm ứng viên danh từ/tính từ trước khi ghép cặp; nhóm bỏ qua để tránh thêm dependency (NLTK/spaCy) không cần thiết cho một baseline tham chiếu.
- **Demo Yelp không có gold label** – độ tin cậy của bảng thống kê trên dữ liệu này chỉ được suy luận gián tiếp từ agreement 97.25% đo trên dữ liệu có nhãn cùng domain (SemEval Restaurant), không phải đo trực tiếp trên chính dữ liệu Yelp.
- **Factual checker mới kiểm chứng ở quy mô demo:** kết quả ở Mục 5.4 đến từ 2 lượt chạy minh hoạ (nhà hàng có nhãn + Yelp quy mô lớn), chưa phải benchmark định lượng trên hàng trăm report – cần chạy trên nhiều mẫu hơn để có số liệu % đáng tin cậy hơn (xem đề xuất ở Mục 8). Checker cũng vẫn là rule-based: dựa vào việc gán mệnh đề cho aspect bằng khớp cụm từ, nên có thể nhầm lẫn với các câu văn phức tạp hơn (nhiều aspect lồng nhau, câu dài nhiều mệnh đề phụ) so với các ví dụ demo hiện tại. Ranh giới `factual_passed/coverage_passed` cũng là một lựa chọn thiết kế có thể tranh luận: coi thiếu aspect là cảnh báo thay vì lỗi chặn (chế độ mặc định `factual-only`) đánh đổi độ đầy đủ của report lấy sự khoan dung hơn với model – phù hợp với mục tiêu ưu tiên chống hallucination, nhưng nghĩa là `passed: true` không còn đảm bảo report nói hết mọi điều đã chọn.
- **Bước tổng hợp gom nhóm theo chuỗi ký tự nguyên văn, không chuẩn hoá hình thái** (phát hiện ở error analysis, Mục 5.5): các biến thể chính tả/số ít-số nhiều của cùng một aspect (`rice to fish ration`/`rice to fish ratio`, `donuts`/`donut`) bị tính thành các aspect tách biệt trong bảng thống kê, làm phân mảnh số liệu và ảnh hưởng gián tiếp tới độ chính xác của báo cáo sinh ra.

8 Kết luận

Nhóm đã xây dựng một pipeline hoàn chỉnh sinh trực tiếp bộ ba khía cạnh–ý kiến–sentiment từ câu thô (AsteLookupBaseline → T5-small → T5-base, cải thiện triplet-F1 gần gấp đôi: 0.3736 → 0.7442), giải quyết đồng thời bài toán trích xuất aspect (không cần gán nhãn trước) và cho thêm trường *opinion* làm “lý do” phục vụ trực tiếp bước sinh báo cáo – với độ nhất quán tổng hợp cao (97.25% majority-sentiment agreement). T5-base là model được khuyến nghị dùng cho bước sinh báo cáo cuối cùng.

Bước sinh báo cáo và kiểm tra tính xác thực cũng cho một tín hiệu tích cực: trên lượt demo thật, FLAN-T5-base đã fine-tune sinh đúng 4/4 claim có căn cứ (Mục 5.4), và factual checker phân biệt được rõ ràng giữa model đã fine-tune (chấp nhận) và model chỉ prompt zero-shot (từ chối đúng lỗi gán nhầm vai trò) – cho thấy cả hai thành phần hoạt động đúng như thiết kế, dù mới kiểm chứng trên quy mô demo, chưa phải benchmark định lượng lớn.

Error analysis (Mục 5.5) không phát hiện thiên lệch hệ thống nào trong 3% case sentiment lệch – toàn bộ đều là các aspect vốn đã sát nút giữa gold và predicted – và xác nhận sai số chính của model nằm ở **recall trên aspect hiếm và đếm thiếu trên aspect phổ biến** (khớp với Precision > Recall đã đo ở Mục 5.1), chứ không phải ở việc suy luận sentiment sai. Phân tích cũng chỉ ra một hạn chế cụ thể, có thể sửa được ở bước tổng hợp (gộp aspect theo chuỗi ký tự nguyên văn thay vì theo hình thái), và đính chính một giả thuyết ban đầu sai (**extract** trong demo Yelp không phải do cắt cụt “vanilla extract” mà là model nhầm lẫn trên câu ngắn/rời rạc) – một ví dụ cho thấy tầm quan trọng của việc kiểm chứng giả thuyết bằng dữ liệu thật thay vì suy đoán.

Hướng phát triển tiếp nếu có thêm thời gian: (1) chuẩn hoá hình thái (lemmatize) ở bước tổng hợp aspect để gộp các biến thể chính tả/số ít-số nhiều, giảm phân mảnh số liệu; (2) thử fine-tune FLAN-T5-base cho ASTE với epoch budget lớn hơn 20 để kiểm chứng trực tiếp giả thuyết “cần nhiều epoch hơn” ở Mục 6; (3) chạy factual checker trên số lượng report lớn hơn để có số liệu tỉ lệ pass/fail đáng tin cậy thay vì một lượt demo; (4) cải tiến **AsteLookupBaseline** bằng POS tagging thật để tách rõ hạn chế do thiếu ngữ pháp khỏi hạn chế do từ vựng giới hạn.

Phân công nhóm

Việc	Thành viên
Data pipeline + baseline	Đạt
Fine-tune BERT/T5 + tổng hợp thống kê	Đạt, Sơn
FLAN-T5 sinh report + factual checker	Hoàng, Vinh, Hưng
Đánh giá & phân tích	Đạt, Hoàng
Viết báo cáo + slides	Đạt, Hoàng, Sơn

A Phụ lục

A.1 Bảng hyperparameter đầy đủ

Model	Epoch	LR	Batch (effective)	Seed
T5-small (ASTE)	20	3e-4	16	42
T5-base (ASTE)	20	3e-4	16	42
FLAN-T5-base (ASTE)	20	3e-4	16	42

Bảng 6: Hyperparameter khớp với config mặc định của repo nền Generative-ABSA [2] (lr=3e-4, batch=16, epoch=20).

A.2 Ví dụ output thực tế

Ví dụ một hàng trong bảng tổng hợp (output/aspect_reasons_restaurant.json), aspect food (827 lượt nhắc, T5-base):

positive (621 lượt): lý do phổ biến nhất – great(109), good(100), delicious(39), excellent(37).
negative (164 lượt): lý do phổ biến nhất – mediocre(10), bad(7), overpriced(5).

A.3 Ví dụ báo cáo tự động sinh ra

Input (4 dòng trích từ output/aspect_reasons_restaurant.json, mảng predicted, rút gọn reason):

```
aspect=food | total=827 | positive=621 | negative=164 | neutral=42 | positive_reasons=great (109), good (100)
aspect=indian food | total=22 | positive=21 | negative=1 | neutral=0 |
positive_reasons=great (5)
aspect=dessert | total=26 | positive=8 | negative=16 | neutral=2 | negative_reasons=not inspired (4)
aspect=waiter | total=42 | positive=24 | negative=18 | neutral=0 | positive_reasons=attentive (6) | negative_reasons=snobby (4)
```

Output của FLAN-T5-base (đã fine-tune) – output/flan_t5_reasoned_report.json:

“The strongest takeaway is the contrast between Indian food and dessert. Indian food was praised in 21 of 22 mentions, especially for being great, whereas dessert received 16 negative comments out of 26, largely citing not inspired. At the same time, food remained the most visible topic, appearing 827 times with 621 positive mentions and great as a recurring reason. Feedback on waiter was less settled: 24 positive versus 18 negative reactions across 42 mentions, with experiences ranging from attentive to snobby.”

Kết quả factual checker: passed: true, claims_checked: 4, valid_claims: 4, missing_aspects: [], unattributed_numbers: [].

Mã nguồn và notebook

- **Repo GitHub của nhóm:** <https://github.com/dattm27/20252B-Gen-AI> – toàn bộ source code, notebook, kết quả (src/, notebooks/, notebooks-output/, output/, plans/project-plan.md).
- **Notebook Kaggle – T5-small (ASTE):** <https://www.kaggle.com/code/sontranvn/train-t5-small-for-aste-on-14res-15res-16res>
- **Notebook Kaggle – T5-base (ASTE):** <https://www.kaggle.com/code/sontranvn/train-t5-base-for-aste-on-14res-15res-16res>
- **Notebook Kaggle – FLAN-T5-base (ASTE):** <https://www.kaggle.com/code/dattm03/train-flan-t5-base-for-aste-on-14res-15res-16r>
- **Notebook Kaggle – GTS-BERT baseline (ASTE):** <https://www.kaggle.com/code/dattm03/gts-bert-baseline-for-aste-on-14res-15res-16re>
- **Notebook Colab – fine-tune FLAN-T5 sinh báo cáo:** <https://colab.research.google.com/drive/1XfBZLL75TTyQEZb5lfyKsofKjYEbqmfF?usp=sharing>

Tài liệu

- [1] Pontiki, M., Galanis, D., Pavlopoulos, J., Papageorgiou, H., Androutsopoulos, I., & Manandhar, S. (2014). *SemEval-2014 Task 4: Aspect Based Sentiment Analysis*. <https://alt.qcri.org/semeval2014/task4/>
- [2] Zhang, W., Li, X., Deng, Y., Bing, L., & Lam, W. (2021). *Towards Generative Aspect-Based Sentiment Analysis*. ACL-IJCNLP 2021. <https://github.com/IsakZhang/Generative-ABSA>
- [3] Peng, H., Xu, L., Bing, L., Huang, F., Lu, W., & Si, L. (2020). *Knowing What, How and Why: A Near Complete Solution for Aspect-Based Sentiment Analysis*. AAAI 2020.
- [4] Xu, L., Chia, Y. K., & Bing, L. (2020). *Position-Aware Tagging for Aspect Sentiment Triplet Extraction*. EMNLP 2020.
- [5] Raffel, C., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research.
- [6] Chung, H. W., et al. (2022). *Scaling Instruction-Finetuned Language Models*. arXiv:2210.11416.
- [7] Kale, M., & Rastogi, A. (2020). *Text-to-Text Pre-Training for Data-to-Text Tasks*. INLG 2020. <https://aclanthology.org/2020.inlg-1.14.pdf>
- [8] Kryscinski, W., McCann, B., Xiong, C., & Socher, R. (2020). *Evaluating the Factual Consistency of Abstractive Text Summarization* (FactCC).
- [9] Wu, Z., Ying, C., Zhao, F., Fan, Z., Dai, X., & Xia, R. (2020). *Grid Tagging Scheme for Aspect-oriented Fine-grained Opinion Extraction*. Findings of ACL: EMNLP 2020. <https://aclanthology.org/2020.findings-emnlp.234/>
- [10] *Consistency Is the Key: Improving Factuality via Consistency-based Checking*. arXiv:2511.12236.